

Purpose

This technical report documents the implementation approach, synchronization algorithm between trackers, piece selection strategy, protocol design rationale, engineering trade-offs, challenges encountered, and references used while developing the assignment submission.

1. Implementation approach

- Language and environment: C++17, POSIX sockets, `std::thread`, OpenSSL SHA1 for hashing.
- Approach: favour clear separation of concerns. Trackers maintain metadata and synchronization; clients handle CLI interaction, peer serving and downloads. Each component is multithreaded with mutex-protected shared state where necessary.

High-level contracts - Tracker - Inputs: textual commands from clients or peer tracker connections. - Outputs: local metadata updates and textual sync commands forwarded to the peer tracker. - Error modes: malformed commands are ignored with logging; numeric parse errors are caught and logged.

- Client
 - Inputs: CLI commands and incoming piece requests from peers.
 - Outputs: peer data responses (`DATA <len>\n<bytes>`), tracker requests for file metadata, and local file writes.
 - Error modes: network failures, verification failures cause retries and are surfaced in logs.

2. Synchronization algorithm

Design goal: Simple, low-overhead eventual replication to keep two tracker nodes consistent for live operations.

Mechanism details - Persistent outbound sync socket: each tracker keeps a persistent TCP client to its configured peer and also accepts inbound sync connections. - Command forwarding: when a local state-changing operation occurs, the same command line is sent over the outbound sync socket. - Loop avoidance: incoming sync commands are applied locally with `is_sync_message=true`, which disables re-forwarding. Accepted inbound sync sockets are tracked and recognized so the server doesn't forward back commands received on that socket.

Properties & limitations - Eventual delivery for live updates while both trackers are up. - No causal ordering or logical clocks — commands are applied in the order they arrive on each tracker. Concurrent conflicting operations may lead to

different outcomes; resolving this requires more sophisticated algorithms (e.g., CRDTs, operation logs with conflicts resolution).

3. Piece selection and download strategy

Goals: simple, balanced load distribution and practical correctness for the assignment.

Details - Piece size: 512 KB. - Strategy: round-robin starting index per piece. For piece index p and peers list `peers[]`, the worker attempts peers starting at `peers[p % peers.size()]` then cycles. This evens out load across peers for different piece indices. - Retry policy: workers have a limited retry count per piece. If a piece fails verification from one peer, the worker tries other peers. If all peers fail, the piece is marked for retry later.

Alternatives evaluated - Rarest-first: requires global availability counts and more messaging; deferred due to scope. - Randomized selection: simpler but can occasionally overload a peer; round-robin is deterministic and predictable.

4. Protocol design rationale

Why text-based? Clarity, simplicity and easy debugging for an assignment environment.

Key constraints handled - Binary payloads: use `DATA <len>\n<raw-bytes>` so the receiver knows exactly how many bytes to read for a piece. - Partial reads: stream processing with an accumulator buffer that extracts newline-terminated commands avoids bugs when TCP delivers partial or combined messages.

Message formats (recap) - Tracker->Client: `FOUND <filesize> <full_sha1> <num_pieces> <filepath_on_peer>\n` + piece hashes + peers list. - Peer->Peer: `get_piece <filepath_on_peer> <piece_idx>\n` and `DATA <len>\n<raw-bytes>` response.

5. Data structures

- Tracker maps:
 - `unordered_map<string, string>` users;
 - `unordered_map<string, unordered_set>` group_members;
 - `unordered_map<string, vector>` group_requests; // FIFO order
 - `unordered_map<string, vector>` group_files; // FileMeta stores per-file meta
 - `unordered_map<string, FileMetaFull>` all_files; // quick global lookup

FileMeta shape

```
struct FileMeta {
    string filename;
```

```

size_t filesize;
string full_sha1;           // hex string
vector<string> piece_hashes; // per-piece SHA1 hex strings
string filepath_on_peer;    // advertised path
};

```

- Client DownloadTask:

```

struct DownloadTask {
    string group_id;
    string filename;
    string dest_path;
    size_t filesize;
    string sha1_full;
    vector<string> piece_hashes;
    vector<uint8_t> piece_status; // PENDING/IN_PROGRESS/DONE
    atomic<int> pieces_done;
    vector<thread> workers;
    atomic<bool> cancel_flag;
    mutex file_write_mutex;
};

```

6. Challenges encountered and solutions

- Partial/fragmented reads
 - Issue: trackers and sync threads originally attempted to parse commands assuming one read equals one command. TCP can deliver partial or multiple commands in one read.
 - Fix: implemented accumulator buffers and loop extracting newline-terminated commands. For DATA <len> payloads the code reads the exact number of bytes following the header.
- Sync forwarding loops
 - Issue: naive re-forwarding of received sync commands caused loops and flooding.
 - Fix: `is_sync_message` flag and tracking accepted inbound sync fds to prevent re-forwarding.
- Numeric parsing and unexpected tokens
 - Issue: `stol/stoi` could throw if a token was malformed.
 - Fix: wrap conversions in try/catch and validate ranges before applying changes.

7. Testing methodology and results

- Unit-level: limited small helper function checks for SHA1 computation.
- Integration: the following manual tests were run repeatedly:
 - Single seeder, single downloader (multi-piece) — verify file SHA1 matches after download.

- Concurrent downloaders (3–4) requesting same file — verify progress and piece distribution.
- Tracker restart: start one tracker, perform uploads, stop tracker, restart — new uploads while offline examined for missing propagation.
- Fault injection: tamper delivered piece bytes at seeder to force verification failure — downloaders retried and succeeded using other peers.

Observed behavior - Parallel downloads improved aggregate throughput for large files. - Piece verification correctly detected corrupt pieces and triggered retries. - Tracker sync successfully applied live updates in typical scenarios; prolonged network partition requires anti-entropy to guarantee perfect replication.

8. Possible improvements and future work

- Durable operation log / anti-entropy: store a short append-only log and perform reconciliation on reconnect.
- Rarest-first selection and availability tracking for better performance at scale.
- Authentication and message signing (HMAC) for tracker sync and peer transfers.
- Replace SHA1 with SHA256 and move to a crypto library wrapper with clear error handling.

How to reproduce the main tests (copy-paste)

1) Build

```
cd Tracker
make
cd ../Client
make
```

2) Start trackers

```
cd Tracker
./tracker ../tracker_info.txt 1
./tracker ../tracker_info.txt 2
```

3) Upload and download

```
cd Client
./client 127.0.0.1:4001 ../tracker_info.txt
create_user seeder pass
login seeder pass
create_group g1
upload_file g1 /full/path/to/test.bin
```

in another terminal

```
./client 127.0.0.1:4002 ../tracker_info.txt  
create_user dl pass  
login dl pass  
join_group g1  
download_file g1 test.bin /tmp/test.bin
```